UR THE INNOVATIVE UNIVERSITY	Challeneg3	Ing4-CYB/CCV	S7

Course Outcomes (Syllabus) avec le pourcentage		
CO	**Exercice**	**%**
CO1: Understand the fundamentals of PP and PA	Part1	30%
CO2: Apply knowledge and develop hands-on skills in developing applications	Part2	30%
CO3: Evaluate parallel code efficiency for HPC tuning and optimization	Part3	40%

Nom[1] :.....**Prénom**:.....

The RSA algorithm is a public-key cryptographic algorithm based on modular arithmetic. In the previous challenges, the RSA mathematical functions were implemented in C, and MPI was used to parallelize the generation of independent RSA key pairs. The resulting project consists of three files:

- **rsa.h**: Declares the RSA functions.
- **rsa.c**: Implements the Euclidean algorithm, extended Euclidean algorithm, modular inverse, primality testing, and RSA key generation.
- **rsa_mpi.c**: Implements the parallel generation of independent RSA key pairs using MPI. Each process generates keys for its assigned tasks by calling *generate_rsa_keys()*.

[1] Tous documents autorisés

The objective of this challenge is to extend the RSA application by implementing parallel text **encryption** and **decryption** using MPI. The new functionality must be implemented in a separate file named **rsa_text_mpi.c**. The application must distribute the characters of a text message among MPI processes, encrypt them independently, collect the encrypted values, and then perform the reverse operation to reconstruct the original message.

The existing files, *rsa.h*, *rsa.c*, and *rsa_mpi.c*, must remain unchanged. All new functionality must be implemented in a separate file named *rsa_text_mpi.c*.

The following tasks needs to be completed:

Part I: Implement RSA text encryption and decryption:

- Create a new file named rsa_text_mpi.c without modifying the existing files.
- Include the necessary header files and reuse the RSA functions already implemented in rsa.h and rsa.c.
- Implement a function to encrypt a character using the public key (N, e).
- Implement a function to decrypt an encrypted character using the private key (N, d).
- Represent each character by its corresponding integer value. For each character, compute its ciphertext using modular exponentiation.
- Ensure that the decrypted characters are converted back to their original representation.

Part II: Parallelize encryption and decryption using MPI

- Initialize the MPI environment and determine the number of processes and the rank of each process.
- Let process 0 define the input text and generate an RSA key pair using the existing generate_rsa_keys() function.
- Distribute the characters of the input text among the MPI processes.
- Assign each process the encryption of its allocated characters independently.
- Collect the encrypted values at process 0 in their original order.
- Distribute the encrypted values among the MPI processes for parallel decryption.
- Collect the decrypted values at process 0 and reconstruct the original text.
- Display the original text, encrypted values, and decrypted text.

Part III: Testing and analysis

- Test the application with different text messages, including messages whose lengths are not divisible by the number of MPI processes.
- Verify that the decrypted text matches the original text exactly.
- Test the application with more MPI processes than characters in the message.
- Measure the execution time for encryption and decryption using different numbers of processes.
- Discuss the distribution of work among processes and the communication overhead introduced by MPI.

For compiling and executing the application using the existing RSA implementation, you might follow:

```
mpicc -Wall -Wextra -c rsa_text_mpi.c -o rsa_text_mpi.o
cc -Wall -Wextra -c rsa.c -o rsa.o
mpicc rsa_text_mpi.o rsa.o -o rsa_text_mpi
mpirun -np 2 ./rsa_text_mpi
mpirun -np 4 ./rsa_text_mpi
mpirun -np 8 ./rsa_text_mpi
```

For an input message such as Challenge, the application must encrypt each character independently, collect the ciphertext values, and decrypt them to recover the original message.